

Lossless Compression of Time Series Data: A Comparative Study

Jonas G. Matt
Automatic Control Laboratory
ETH Zürich
Zürich, Switzerland
jmatt@ethz.ch

Pengcheng Huang
Corporate Research Center
ABB
Baden-Dättwil, Switzerland
pengcheng.huang@ch.abb.com

Balz Maag
Corporate Research Center
ABB
Baden-Dättwil, Switzerland
balz.maag@ch.abb.com

Abstract—Our increasingly digital and connected world has led to the generation of unprecedented amounts of data. This data must be efficiently managed, transmitted, and stored to preserve resources and allow scalability. Data compression has therein been a key technology for a long time, resulting in a vast landscape of available techniques. This largest-to-date study analyzes and compares various lossless data compression methods for time series data. We present a unified framework encompassing two stages: data transformation and entropy encoding. We evaluate compression algorithms across both synthetic and real-world datasets with varying characteristics. Through ablation studies at each compression stage, we isolate the impact of individual components on overall compression performance – revealing the strengths and weaknesses of different algorithms when facing diverse time series properties. Our study underscores the importance of well-configured and complete compression pipelines beyond individual components or algorithms; it offers a comprehensive guide for selecting and composing the most appropriate compression algorithms tailored to specific datasets.

Index Terms—compression, time series, delta coding, quantile transform, integer compression

I. INTRODUCTION

The proliferation of time series data in today's digital landscape reaches unprecedented levels. For the industrial sector, the rapid emergence of Industry 4.0 has catalyzed Internet of Things (IoT) technologies, resulting in an exponential increase in data generation rates. Estimates from recent studies suggest that in 2025, IoT devices alone will generate more than 180 zettabytes (10^{21} bytes) of new data per year [1]. Reducing the amount of this data is paramount to maximize the efficiency of its transmission, processing, and storage.

One of the most general and effective approaches for reducing data volume is lossless data compression. Lossless compression exploits statistical redundancy in the data to represent it more efficiently but at no loss of information. It stands out from other data reduction techniques, such as lossy compression, decreasing sampling frequency or numerical accuracy, because it can be applied independently without compromising the data being collected.

State-of-the-art lossless compression techniques were designed primarily for text or unstructured binary data rather than numeric datasets [2]–[5]. In fact, the most relevant compression benchmarks used today focus almost exclusively on text

data [6], [7], leaving out important aspects related specifically to compressing numeric sequences. This has sparked the proposal of new methods in recent years [8]–[12]. Yet, in the authors' opinion, compression techniques tailored to such data remain understudied. Existing works on the compression of time series either appear as pure surveys of literature [13], [14], or they propose new methods including comparisons against other baseline algorithms [8]–[12], [15]. Neutral comparisons the authors are aware of are limited in the number and variety of algorithms they feature [16], [17] and/or were designed with specific applications in mind [18]–[20].

Hence, there is a clear need for an extensive systematic comparison of existing methods to provide guidance for choosing appropriate algorithms. This paper presents the largest-to-date comparison of lossless compression algorithms applicable to time series. We include both general-purpose compression algorithms as well as new methods targeted at time series. We provide a common framework encompassing those techniques and identify the key steps involved within time series compression pipelines. By ablating the most common building blocks of such pipelines, we highlight the impact of various components given different signal characteristics. A special emphasis is put on evaluating the effects of compression-aiding data transformations such as delta coding, which are employed before the actual data compression. By granting equal access to these transformations, fair assessments can be performed to conclude the impact of various methods adopted in different stages of the compression pipelines, which are often under-represented in the literature. Overall, our research highlights the importance of well-configured and holistic compression pipelines, surpassing the focus on individual components or algorithms. Our study provides a comprehensive framework and guidance for selecting and integrating the most suitable compression algorithms for specific time series datasets.

The rest of this paper is organized as follows: In Section II, the necessary notation and metrics are defined, and the compared compression methods are introduced. Section III describes the proposed framework for compression pipelines. In Section IV, we present the results of the comparative study, and we conclude and summarize the paper in Section V.

II. DEFINITIONS AND METHODS

A. Time Series

An (integer-valued) time series $\mathcal{T}$ is a chronologically ordered sequence of n samples $\mathcal{T} := (x_k)_{k=1}^n = (\mathbf{x}_1, \dots, \mathbf{x}_n)$ obtained from the same source, where $\mathbf{x} \in \mathbb{Z}^d$. If $d = 1$, the time series is called univariate, and multivariate otherwise. Note that industrial time series from sensor readings can often be “integerized”, as the original readings are performed by analog-to-digital converters with fixed numbers of total values. This is why we focus on integer time series in this study.

1) *Delta of a time series*: The delta of a time series yields another time series that contains the differences between consecutive samples of the original time series. Formally, the delta of a time series $(x_k)_{k=1}^n$ is defined as $(\Delta x_k)_{k=1}^n := (x_1, x_2 - x_1, \dots, x_n - x_{n-1})$. One can view delta coding as a common decorrelation technique for time series data, allowing to transmit only new information or relative entropy.

2) *Average absolute deviation and cardinality*: We define two distinct measures for the “spread” of a time series: its average absolute difference (AAD) and its cardinality. The AAD is the average of the absolute deviations of a time series’ samples from some center value. For our purpose, we define the center value as 0. The cardinality is the number of distinct values in the time series; it serves as a measure of a signal’s information or entropy.

3) *Compression ratio and score*: Arguably, the most important aspect of any compression method is its ability to shrink the size of the data. A straightforward way to measure this is the compression ratio $\text{CR} := \frac{\text{original size}}{\text{size after compression}}$, with the size of both the source and encoded data measured in bytes. By definition, the compression ratio is unbounded. To facilitate the direct comparison of methods in a more intuitive manner, we define the compression score as $\text{CS} := 1 - \frac{1}{\text{CR}}$. The compression score can be interpreted as the fraction of the file size removed by compression.

4) *Compression time and speed*: Comparing the computational complexity of different compression algorithms experimentally is more difficult than comparing their compression abilities. A fair comparison is impeded by differences in implementation and other external factors. Nevertheless, a common approximation of a compression method’s time complexity is compression time. It is defined as the computation time in seconds that a given algorithm requires to compress a given data. Further, we define the compression speed as the megabytes of data compressed in one second, with unit MB/s. We refrain from making any statements about memory requirements of the reviewed methods because we cannot guarantee a fair comparison.

B. Compression Algorithms

We briefly review state-of-the-art lossless compression algorithms applicable to time series. We provide a unified framework in Section III to compare those algorithms. All reviewed algorithms are open source.

Entropy coding: As a fundamental technique in compression, entropy coding leverages the statistical properties of data

to assign shorter codes to more frequent symbols, thereby reducing the overall data size. It aims to approach the theoretical limit of compression efficiency as defined by Shannon’s source coding theorem [21].

1) *Huffman coding* [22]: This is an optimal prefix coding algorithm widely used in lossless data compression. It begins by creating a priority queue of nodes, each representing a symbol and its frequency. The two nodes with the lowest frequencies are repeatedly merged to form a binary tree. The path from the root to each leaf node determines the symbol’s binary code, with shorter codes for more frequent symbols.

2) *Arithmetic coding* [23]: Instead of assigning fixed codes to individual symbols, arithmetic coding represents the entire message as a single number between 0 and 1. Starting with the interval $[0,1)$, each arriving symbol refines the interval based on its probability, assigning larger subintervals to more frequent symbols. This process continues iteratively until the final interval encodes the entire message.

3) *Asymmetric numeral systems* [24]: First proposed by Jarosław Duda in 2009, Asymmetric Numeral System (ANS) are a modern family of entropy coders. ANS guarantee compression close to the Shannon limit for all distributions, all while having a time complexity similar to Huffman coding.

Dictionary coding: This category of classical compression algorithms works by representing repetitive patterns in data with dictionaries.

1) *LZSS* [25]: Lempel-Ziv Storer-Szymanski (LZSS) is one of the most popular variants of the Lempel-Ziv (LZ) algorithm family. It improves on the basic LZ77 [26] and LZ78 [27] algorithms by omitting dictionary references that are longer than the sequence they would replace. Like most general-purpose algorithms, LZSS acts on the source data in a byte-wise manner and is agnostic to the underlying data type.

2) *LZ4* [28]: As a modern algorithm based on LZ dictionary coding, LZ4 focuses on compression speed. By utilizing the capabilities of modern CPUs, it can achieve more than 500 MB/s per core during compression and multiple GB/s per core during decompression.

3) *Snappy* [29]: Snappy was developed by Google as another algorithm for high-speed requirements. Like LZ4, it is a modern reinterpretation of the LZ algorithms.

Hybrid general-purpose methods: This class of methods combines multiple stages to achieve better compression, following the framework discussed in Section III.

1) *zlib* [30]: One of the most popular hybrid methods is the Deflate algorithm [2]. It consists of a dictionary coding stage based on LZ77 followed by Huffman coding. Deflate is among the most widely used compression algorithms and is part of popular archivers such as ZIP and 7-Zip; zlib is a popular implementation of Deflate.

2) *LZMA* [31]: LZMA combines LZ77-like dictionary coding with range coding, an implementation of arithmetic coding. Owed to its arithmetic coding stage, LZMA requires more memory and compresses more slowly than Deflate.

3) *Brotli* [5]: Brotli is a general-purpose compression algorithm developed by Google and released in 2013. It combines

a modern version of LZ77 dictionary coding with Huffman entropy coding and second-order context modeling. Various optimizations enable it to achieve better compression ratios than Deflate, at similar speeds and memory requirements.

4) *Zstandard* [4]: With the emergence of ANS as a new type of entropy coder, their usage in hybrid methods was explored by, e.g., Facebook and gave rise to Zstandard (Zstd). It is based on Yann Collet's Finite State Entropy (FSE) [32], a tabled implementation of ANS. Overall, Zstd combines an LZ77 dictionary coder with a two-stage entropy coder featuring both Huffman coding and FSE.

5) *bzip2* [3]: In contrast to the other hybrid methods, bzip2 does not feature a dictionary coder. Instead, it applies a sequence of run-length encoding (RLE) [33], a Burrow-Wheeler transform [34] (BWT), a move-to-front transform (MTF) [35], and another RLE. The output of the final RLE is Huffman-coded.

6) *Blosc* [36]: Blosc is a lossless compression algorithm primarily designed for scientific computing and large datasets. It employs a blocking strategy to split data into chunks that are independently shuffled and compressed. Using multi-threading, Blosc achieves low latency and high throughput.

Integer compression: Specialized methods for integer compression exploit the separation of numbers at their actual byte boundaries (e.g., taking two bytes at a time for 16-bit integers). Then, they aim to store each number as efficiently as possible given their distribution. Most methods make an implicit assumption about the numerical distribution of the data. Most commonly, smaller numbers are assumed to occur more frequently than larger numbers. Thus, compression is achieved by assigning shorter codes to smaller numbers. We refer to this as the “smaller-numbers-fewer-bits” principle.

1) *Exp-Golomb* [37]: Exponential Golomb is a universal code that combines unary and binary coding; it represents a number n as a sequence of zeros followed by the binary representation of $n+1$. The length of the zero sequence is one less than the length of the following binary representation.

2) *Bit packing*: Bit packing aims at storing integers at the minimum number of bits required. The number of bits is determined by the size of the largest integer in a block of data. For each block, the number of bits is stored once, followed by the integers in that block. An example of bit packing algorithms is the Simple family [38].

Time series compression: Recently, a few specialized methods have been proposed for the compression of time series.

1) *DRH* [8]: DRH proposes to compress time series via the three following stages: delta coding, RLE, and a static Huffman code that is inspired by the JPEG standard [39]. Delta coding and run-length encoding make up the decorrelation stage and the static Huffman code is the entropy coding stage. We use a custom implementation of the static Huffman code.

2) *Sprintz* [10]: Sprintz is a time-series-specialized compression algorithm that employs a combination of delta coding, run-length encoding, and bit-packing. A forecasting-based encoding strategy called FIRE (Fast Integer REgression) may be used in place of delta coding. Since the improvements of

FIRE are small (according to the authors of Sprintz and our own experiments), we omit this variant of Sprintz. Focussing on delta coding also fits the ablation study of this paper.

3) *Pcodec* [40]: Pcodec is an open-source compression method designed for numerical sequences. It uses a three-step procedure: (1) mode decomposition to split numbers into latent variables based on their approximate structure, (2) delta coding to compress differences between consecutive elements, and (3) quantile-based binning, its core innovation, to entropy-code latent variables into bins paired with exact offsets. Its binning strategy enables Pcodec to approach the Shannon limit for any smooth numerical distribution.

C. Methods considered but not included

The following methods were considered but did not find their way into the experiments of this paper.

1) *FastPFor* [41]: A library for fast integer compression that supports a variety of different codecs. However, it is limited to 32-bit integers.

2) *Titchy* [11]: A recently proposed time series compression algorithm that is specifically designed for random access but is without an official implementation.

3) *Gorilla* [42]: A time series compression algorithm designed for floating point data. Due to the generally lower compressibility of floats, it is not competitive on integer data.

4) *Other universal codes*: We omit other static integer codes such as Rice [43] or Fibonacci [44] codes as they would not yield additional insights next to Exp-Golomb and DRH.

III. A UNIFIED FRAMEWORK FOR COMPRESSION PIPELINES

We have seen in Section II that there is a plethora of lossless compression algorithms applicable to time series data, all following different design principles and even encompassing different modules/components within a single compression algorithm. A unified framework for compression is therefore needed to (i) identify the common components of compression algorithms, (ii) perform ablation study of such components and study individually their impacts on overall compression, (iii) fairly compare different compression algorithms by giving them access to common fundamental components and (iv) guide the composition of new compression pipelines by combining different building blocks within this unified framework.

Our proposed unified compression framework consists of two stages, compression-aiding transformations, and entropy coding. For each stage, many different algorithms exist following possibly drastically different design philosophies.

Entropy coding aims at finding the most efficient bit representations (codes) for the source symbols, achieving compression close to the data's entropy. Entropy coders represent the source symbols independently and implicitly assume that they are independent and identically distributed (i.i.d.). However, real data often has internal correlations, leading to redundancy. Reducing such redundancies, or entropy of the data in more theoretical terms, is thus helpful in enhancing the performance of entropy coders. Compression-aiding transformations aim to reduce the amount of information to be encoded or to prepare

	Transformations	Entropy coding	Class
Huffman		Huff	Entropy only
LZSS	LZ		Dictionary only
Snappy	LZ		
LZ4	LZ		
Zstandard	LZ	Huff	Hybrid general purpose
Brotli	LZ	Huff ANS	
zlib / Deflate	LZ	Huff	
bzip2	RLE BWT MTF	Huff	
LZMA	LZ	Range	
DRH	Delta RLE	Huff	Time series
Sprintz	Delta RLE	Bitpacking	
Pcodec	Delta RLE	Quantile binning	

Compression →

Fig. 1. Overview of the main compared methods categorized by the unified compression framework, detailing their transformation and entropy coding stages.

the data such that it can be better encoded by certain entropy coders.

Figure 1 presents a summary of selected representative compression algorithms as discussed in Section II, according to our compression framework. The two-stage concept is the foundation of existing hybrid methods [2], [4], [5], [30] as well as methods tailored to time series data [8], [10], [12].

With Figure 1, we provide a more complete picture of how new tailored compression methods could be constructed for specific use cases. In the following, we discuss the two stages and some common design principles employed.

A. Compression-aiding transformations

The following transformations (among others) are applied before entropy coding to convert internal correlations into higher compression ratios.

1) *Dictionary coding*: For many types of data, the correlation between source symbols manifests itself in the recurrence of identical sequences of symbols. Occurrences of the same word in a text are just one example, occurrences of identical values in a time series are another. Dictionary coding techniques enhance compression performance by removing such redundancy. They construct a dictionary of recurring symbol sequences and replace source sequences with their dictionary positions. Implementations such as LZ77 and its variants (LZ78, LZSS, LZW) [25]–[27], [45] use a sliding window to store substrings of processed data, limiting memory usage. Matches in the sliding window are replaced by their relative positions and lengths.

2) *Delta coding*: Not without reason, delta coding is part of virtually all compression methods that specialize in time series [8], [10], [12]. It decorrelates data by transforming it into a series of differences between consecutive values, rather than the values themselves. This process reduces the correlation between data points, making the data more uniform and predictable. By encoding the differences (deltas) instead

of the actual values, delta coding captures the small variations between data points, reducing redundancy.

3) *Run-length encoding* [33]: A feature commonly found in real data is consecutive repetitions of the same symbol. In a time series, this corresponds to a signal that remains constant for some time. RLE replaces such “runs” by a pair consisting of the symbol itself and the number of repetitions. Given some prior knowledge of the data, it can be beneficial to apply RLE only to a subset of source symbols. For instance, in the case of time series deltas, it makes sense to apply RLE only to runs of zeroes. This is the variant of RLE used in our experiments.

4) *Quantile reshuffling* [46]: Quantile Reshuffling (QuaRs) aims at facilitating compression techniques that encode smaller numbers (smaller AAD) with fewer bits. It reshapes arbitrary numerical data distributions into unimodal distributions centered around zero; during the reshuffling, more frequent numbers (ordered by quantile bins) are mapped to smaller values. This way, it effectively reduces the AAD of a dataset.

B. Entropy coding

Once data is preprocessed with potential entropy-reducing and compression-aiding transforms, the entropy coding stage then tries to find the most efficient codes or bit representations for the symbols in the transformed data. It was the traditional focus of compression research and covers a wide variety of algorithms. Existing methods can be categorized as follows.

1) *Dynamic codes*: These codes are designed based on a given data distribution. Some information on the data’s statistical properties is also required for decoding, and must hence be transmitted along with the compressed data. Concrete methods include Huffman coding, arithmetic coding, and ANS.

2) *Universal codes*: In contrast, universal codes are designed to efficiently encode integers without prior knowledge of data distribution. Typically, they assign shorter codes to smaller integers, following the smaller-numbers-fewer-bits principle. Common methods include Fibonacci coding [44], Exp-Golomb coding, and Elias codes [47]. Each method can be associated with an induced probability distribution, for which it represents an optimal prefix code.

IV. EXPERIMENTS

The following experiments evaluate the algorithms described in Section III on various types of time series data. In the following Section (IV-A), we state the specifications of the experimental setup. In Section IV-B, synthetic data is used to demonstrate the isolated effect of different signal characteristics on compression performance. Section IV-C presents the results of a comparison on real-world datasets.

A. Experimental setup

The experiments are conducted on a Linux machine with an Intel Xeon Silver 4208 CPU (2.10 GHz) and 96 GB of RAM. Unless stated otherwise, we use the methods at their highest or close-to-highest settings: Compression level 9 for zlib, level 19 for Zstd, level 10 for Brotli, level 9 for bzip2, level 6 for LZMA, level 9 and BloscLZ as the dictionary coder for Blosc.

We use delta coding as the forecaster for Sprintz, and level 12 and no higher-order delta coding for Pcodec. For LZSS, we use a buffer size and lookahead buffer size of 4096 and 18 bytes, respectively.

B. Synthetic time series data

We start by investigating the impact of isolated signal characteristics on compression performance. We focus on a comparison of compression scores in this section and defer the discussion of compression speeds to Section IV-C. In both sections, we number our main findings to enhance readability.

Real data can be interpreted as a superposition of various basic signal characteristics. With this in mind, we use a suite of synthetic time series to represent a selection of these basic characteristics. Thus, each test case represents one particular signal characteristic that may occur in time series data. The selected signals are as follows:

1) *Sine*: An example of a signal that varies slowly relative to the sampling frequency. This is where delta coding is the most effective.

2) *Noise*: Uniformly and independently distributed random noise. Such data has the maximum theoretical entropy, making it the worst case for compression.

3) *Sine + noise*: The sum of the previous two signals. This case illustrates noise can be a bottleneck for compression.

4) *Switching*: A signal switching between a discrete set of values at short intervals. This yields a sparse signal with low cardinality but high AAD. It is a realistic failure case for algorithms that rely on the efficient encoding of small integers.

Fig. 2 depicts the synthetic data along with its transformed variants (delta coding, QuaRs). Table I lists their cardinality and AAD, before and after applying the transformations. The

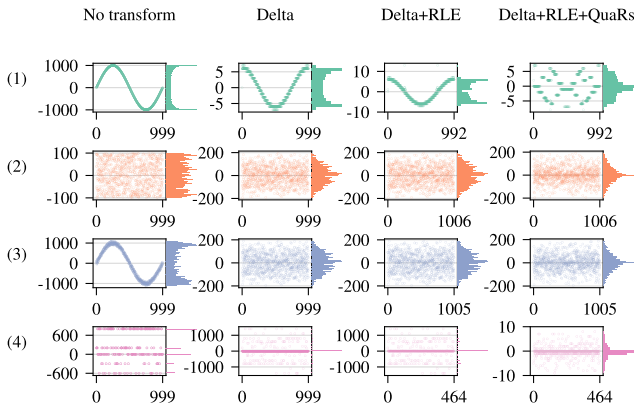


Fig. 2. The synthetic test cases, before and after progressively applying the different transformations (delta coding, RLE, QuaRs). This demonstrates the impact of each transformation stage on data representation.

cardinality and AAD of a dataset are good indicators of its compressibility. Observing how transformations affect these metrics helps understand their effectiveness. Cardinality has a direct impact on entropy and, hence, lower cardinality generally suggests better compressibility. Meanwhile, low AAD is a relevant compressibility indicator only for methods that operate according to the smaller-numbers-fewer-bits principle.

TABLE I
CARDINALITY / AAD OF THE SYNTHETIC TIME SERIES BEFORE AND AFTER PROGRESSIVELY APPLYING THE TRANSFORMATIONS. D, R, AND Q DENOTE DELTA CODING, RLE, AND QUARS, RESPECTIVELY.

	No transform	D	D+R	D+R+Q
Sine	897 / 635.5	15 / 4.0	16 / 4.1	16 / 2.7
Noise	196 / 49.7	313 / 65.0	313 / 64.5	313 / 52.1
Sine + Noise	789 / 639.0	316 / 65.5	316 / 65.2	316 / 53.6
Switching	5 / 399.0	15 / 106.4	18 / 230.6	18 / 1.6

Among the reviewed methods, this is the case for Golomb coding, DRH, and Sprintz.

(1) *If the values of a time series are sufficiently autocorrelated (the time series is “slowly varying”), delta coding can significantly reduce the entropy of the data.* In the sine and sine+noise test cases, this manifests in a reduction of both AAD and cardinality. The other two test cases exhibit much less autocorrelation and delta coding actually increases the cardinality and AAD. (2) *QuaRs does not change the cardinality of the data but reduces the AAD.* QuaRs reorders bins of the data such that the most frequent values become the smallest. It is particularly effective for the switching data, for which it reduces the AAD by a factor of almost 200.

(3) *Run-length encoding is most effective on data with long runs of constants.* This is the case for the switching test case, where RLE reduces the length of the data by more than half (see the x-axis in Fig. 2).

Fig. 3 shows the compression scores achieved by all methods on the test case data. To follow the markers along lines from left to right corresponds to first adding a delta coding, then RLE, and finally a QuaRs transformation stage before compression. DRH, Sprintz, and Pcodec incorporate a delta coding stage by design. We actively disable this stage in their source code to obtain results without delta coding for illustrative purposes.

(4) *The Shannon limit is inversely proportional to the entropy and, thus, the cardinality of the data.* The leftmost column in Fig. 3—indicated by $H(x)$ —represents the Shannon limit [21]. This is the theoretical upper bound for the compression achievable by any method, given that the data is uncorrelated. By exploiting correlation in the data to decrease its cardinality, delta coding can “improve the Shannon limit”, i.e., decrease the pointwise entropy of the data. This directly leads to better compressibility.

QuaRs, on the other hand, does not change the cardinality of the data and hence does not affect the Shannon limit. However, it can still improve compression scores by reducing the AAD. This can best be observed in the switching test case, where QuaRs improves the compression scores of Golomb, DRH, and Sprintz by 18%, 13%, and 21% respectively. All the other methods are negligibly affected by QuaRs, as they do not benefit from the decrease in AAD.

(5) *Noisy signal components are a bottleneck for compression.* If a signal is purely noise, no correlation can be exploited for better compression, rendering the Shannon limit the true upper bound for the compression achievable by any

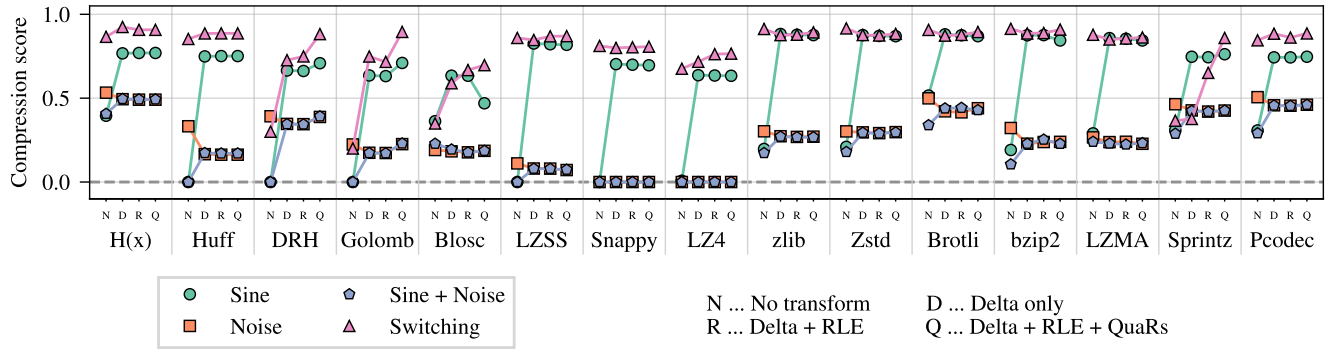


Fig. 3. Compression scores achieved by the different methods on the synthetic test cases. The markers along a line correspond to the progressive application of delta coding, RLE, and QuaRs. Compression scores below 0 (compressed data is larger than the original data) are shown as 0.

method. In the sine+noise test case, delta coding effectively removes the low-frequency component (the sine), leaving just the noise. Consequently, the Shannon limit (and the achieved compression scores) in the noise and sine+noise test cases are identical after delta coding.

(6) *Overall, the hybrid and time-series-specialized methods yield the best compression scores.* Among the general-purpose compressors, the hybrid methods are the best-performing ones. Brotli is the best in this category. Generally, the hybrid methods—zlib, Zstandard, Brotli, LZMA, and bzip2—outperform the dictionary-coding-only methods LZSS, Snappy, and LZ4. This shows that deploying an entropy coding stage is crucial for achieving good compression.

(7) *Sprintz achieves competitive compression ratios except on certain types of data.* Decorrelating transforms such as QuaRs can help to mitigate these shortfalls. (8) *Pcodec is a promising allrounder method tailored to time series.* It achieves competitive compression ratios across all sorts of time series characteristics.

(9) *Simple static codes such as Exp-Golomb can achieve decent compression ratios on various time series, if they are paired with suitable transformations.*

(10) *Dynamic codes such as Huffman are theoretically optimal, but the required headers quickly become a bottleneck if the data has high cardinality.* This can be observed in the sine and sine+noise test cases, for which Huffman coding falls short of the Shannon limit due to large headers.

C. Real-world time series data

To obtain a performance evaluation that is closer to industrial practice and to conduct a comparison of compression speeds, we compare the compression algorithms on a selection of real-world time series datasets. Except for ABB Drive, the datasets are open-source, inviting replication of the presented results. They have been selected to represent a variety of domains.

Time series datasets are typically distributed in floating point format although in most cases, this is not how the data was originally represented (cf. II-A). We re-quantize the data by scaling the data to the maximum range of the chosen

data type (16-bit integers) and applying the floor function. This yields virtually no loss of precision for all datasets. The datasets are as follows:

1) *UCR Time Series Classification Archive [48]:* The UCR Archive is a collection of 128 different univariate signals. Its intended use is as a benchmark for time series classification.

2) *PAMAP2 [49]:* A dataset featuring heart rate and IMU measurements from sensors on human subjects performing various physical activities.

3) *AMPds2 [50]:* The Almanac of Minutely Power dataset (Version 2) contains measurements of minutely electricity, water, and natural gas consumption of a single home and over a span of two years.

4) *ABB Drive:* A dataset containing measurements from a drive system in an industrial setting.

5) *MSRC-12 [51]:* The MSRC-12 Kinect Gesture Dataset contains tracking data of body-part locations.

Fig. 4 summarizes the compression scores and speeds achieved by the different methods on these datasets. For these results, all methods are used as-is, i.e. Sprintz and Pcodec incorporate delta coding but no additional transformations are applied along with any method. In contrast, Table II showcases the impact of delta coding and QuaRs on the compression scores for some selected methods. We showcase the effect of QuaRs for Sprintz, which is one of the methods that benefit most from a small AAD. Furthermore, we demonstrate the effect of delta coding and QuaRs for Brotli and bzip2, which are the best-performing general-purpose methods. Finally, we omit any results for Huffman, Golomb coding, and DRH because their available *implementations* are not competitive in terms of speed.

The results for the UCR, PAMAP2, and MSRC-12 datasets are qualitatively similar. The highest compression scores achieved are around 40-55% for these datasets. (1) *Brotli, LZMA, and bzip2 achieve good compression scores but exhibit the slowest speeds*, clustering in the lower-right portion of the graph. Snappy and LZ4 occupy the opposite side of the graph; they compress very fast but achieve only a slight reduction of the data size. (2) *The two time-series-specialized methods Sprintz and Pcodec achieve the best trade-off of the two*

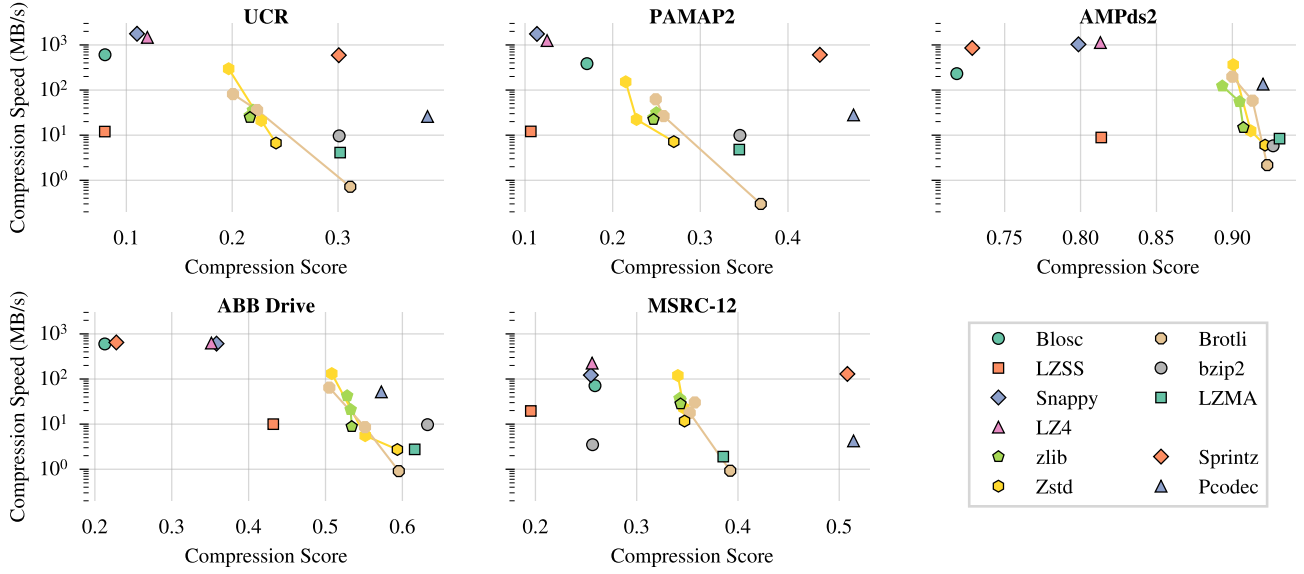


Fig. 4. Compression scores and speeds achieved by the different methods on the real-world datasets. Markers with no outline indicate results using compression level settings different from the default ones stated in Section IV-A. They indicate the speed-compression trade-off achievable by the methods Zstd, Brotli, and zlib.

TABLE II
THE IMPACT OF DELTA CODING AND QUARs ON THE COMPRESSION SCORES FOR SELECTED METHODS.

		No transform	Delta	Delta + QuaRs
UCR	Brotli	0.31	0.43	0.41
	bzip2	0.30	0.41	0.39
	Sprintz		0.30	0.34
PAMAP2	Brotli	0.37	0.48	0.48
	bzip2	0.35	0.47	0.46
	Sprintz		0.44	0.46
AMPds2	Brotli	0.92	0.92	0.92
	bzip2	0.93	0.93	0.93
	Sprintz		0.73	0.87
ABB Drive	Brotli	0.60	0.56	0.56
	bzip2	0.63	0.59	0.58
	Sprintz		0.23	0.45
MSRC-12	Brotli	0.39	0.51	0.51
	bzip2	0.26	0.37	0.37
	Sprintz		0.51	0.51

metrics. Among the two, Sprintz is the faster one, achieving speeds comparable to Snappy and LZ4 and compression scores similar to the best general-purpose methods. Pcodec is slower than Sprintz but faster than Brotli, LZMA, and bzip2—and compresses better than any other method. Zstd, zlib, LZSS, and Blosc lie below the Pareto front, yielding strictly worse trade-offs than other methods.

In Table II, one can observe the effect of the transformations on compression performance. (3) *Delta coding is a crucial component for achieving good compression scores.* State-of-the-art general-purpose methods clearly benefit from a delta coding stage. As can be seen in Table II, the compression scores of Brotli and bzip2 rise by more than 10% for the UCR and PAMAP2 datasets. This illustrates why methods such as DRH, Sprintz, and Pcodec incorporate delta coding by design.

For the AMPds2 and ABB Drive datasets, a few differing observations can be made. These datasets are characterized by

long runs of constants, manifests in a low cardinality but high AAD. Consequently, they are generally more compressible than the other datasets; compression scores of up to 93% for AMPds2 and 63% for ABB Drive are achieved. At the same time, their characteristics make these datasets realistic failure cases for some compression algorithms that rely on the efficient encoding of small integers. Sprintz—being one such method—is in fact not competitive on its own on this dataset. However, this is also a scenario in which QuaRs is highly effective (as discussed in IV-B). (4) *After applying QuaRs, Sprintz achieves compression scores on par with the best-performing methods.* Hence, combining QuaRs and Sprintz enables competitive compression performance at unprecedented speeds.

V. CONCLUSION

We presented a comprehensive comparison of lossless time series compression algorithms using a two-stage framework of data transformation and entropy encoding. By ablating stages and conducting the largest evaluation to date, we identified key components that drive compression performance.

Our results highlight the importance of holistic pipelines, as seen in the strong performance of hybrid compressors like bzip2, Brotli, LZMA, and specialized methods like Sprintz and Pcodec. We further showed that applying transformations such as delta coding and QuaRs can improve compression across a variety of settings. However, no single algorithm universally outperforms others across all datasets.

The insights from this study can guide practitioners in method selection and motivate further work toward more adaptable and efficient time series compression approaches.

VI. FUTURE DIRECTIONS

Our findings point to several directions for future research: (1) *Benchmarking efforts:* Future benchmarking should evalu-

ate full compression pipelines, emphasizing the importance of both transformation and encoding stages. High-quality, user-friendly implementations are crucial for ensuring consistent comparisons and encouraging broader adoption. Unfortunately, this is an area that still needs improvement at the time of our study. (2) Scope expansion: Several aspects lie beyond the scope of our study, including lossy compression and streaming applications, both of which pose distinct challenges and opportunities. Future work should also consider the memory requirements and embedded-friendliness of compression methods, as well as their random access capabilities. Comprehensive comparisons along these dimensions are difficult but would be beneficial for the community.

REFERENCES

- [1] "Data growth worldwide 2010-2025," <https://www.statista.com/statistics/871513/worldwide-data-created/>.
- [2] L. P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," Tech. Rep. RFC 1951, Internet Engineering Task Force, May 1996.
- [3] J. Seward, "Bzip2," <https://sourceware.org/bzip2/>, 2019.
- [4] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," Tech. Rep. RFC 8878, Internet Engineering Task Force, Feb. 2021.
- [5] "Google/brotli," Google, July 2024.
- [6] M. Mahoney, "Large Text Compression Benchmark," <https://mattmahoney.net/dc/text.html>, 2011.
- [7] "Inikep/lzbench," <https://github.com/inikep/lzbench>.
- [8] H. S. Mogahed and A. G. Yakunin, "Development of a Lossless Data Compression Algorithm for Multichannel Environmental Monitoring Systems," in *2018 XIV APEIE*, (Novosibirsk), pp. 483–486, IEEE, Oct. 2018.
- [9] J. Spiegel, P. Wira, and G. Hermann, "A Comparative Experimental Study of Lossless Compression Algorithms for Enhancing Energy Efficiency in Smart Meters," in *2018 IEEE 16th International Conference on Industrial Informatics (INDIN)*, (Porto), pp. 447–452, IEEE, July 2018.
- [10] D. Blalock, S. Madden, and J. Gutttag, "Sprintz: Time Series Compression for the Internet of Things," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, pp. 1–23, Sept. 2018.
- [11] R. Vestergaard, Q. Zhang, M. Sipos, and D. E. Lucani, "Titchy: Online Time-Series Compression With Random Access for the Internet of Things," *IEEE Internet Things J.*, vol. 8, pp. 17568–17583, Dec. 2021.
- [12] M. Loncaric, "Pcodec: Lossless codec for numerical data," 2023.
- [13] G. Chiarot and C. Silvestri, "Time Series Compression Survey," *ACM Comput. Surv.*, vol. 55, pp. 1–32, Oct. 2023.
- [14] M. A. De Oliveira, A. M. Da Rocha, F. E. Puntel, and G. G. H. Cavalheiro, "Time Series Compression for IoT: A Systematic Literature Review," *Wireless Communications and Mobile Computing*, vol. 2023, pp. 1–23, Aug. 2023.
- [15] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa, "DZip: Improved general-purpose lossless compression based on novel neural network modeling," Sept. 2020.
- [16] F. Bjärås, "Comparative study of compression algorithms on time series data for IoT devices," Master's thesis, Lund University, Sweden, July 2019.
- [17] J. Hughes, "Comparison of lossy and lossless compression algorithms for time series data in the Internet of Vehicles," Master's thesis, Linköping University, Sweden, 2023.
- [18] K. L. Ketshabetswe, A. M. Zungeru, B. Mtengi, C. K. Lebekwe, and S. R. S. Prabaharan, "Data Compression Algorithms for Wireless Sensor Networks: A Review and Comparison," *IEEE Access*, vol. 9, pp. 136872–136891, 2021.
- [19] M. Pasetti, E. Sisinni, P. Ferrari, P. Bellagente, and D. Zaninelli, "Comprehensive evaluation of lossless compression algorithms in a real use case for smart grid applications," *Sustainable Energy, Grids and Networks*, vol. 36, p. 101238, Dec. 2023.
- [20] I. Bozhilov, R. Petkova, K. Tonchev, and A. Manolova, "A Systematic Survey Into Compression Algorithms for Three-Dimensional Content," *IEEE Access*, vol. 12, pp. 141604–141624, 2024.
- [21] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, pp. 379–423, July 1948.
- [22] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, pp. 1098–1101, Sept. 1952.
- [23] A. Said, "Introduction to Arithmetic Coding – Theory and Practice," Feb. 2023.
- [24] J. Duda, "Asymmetric numeral systems," May 2009.
- [25] J. A. Storer and T. G. Szymanski, "Data compression via textual substitution," *J. ACM*, vol. 29, pp. 928–951, Oct. 1982.
- [26] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inform. Theory*, vol. 23, pp. 337–343, May 1977.
- [27] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Trans. Inform. Theory*, vol. 24, pp. 530–536, Sept. 1978.
- [28] Y. Collet, "LZ4 - Extremely fast compression," 2014.
- [29] "Snappy: A fast compressor/decompressor," Google, 2014.
- [30] L. P. Deutsch and J.-I. Gailly, "ZLIB Compressed Data Format Specification version 3.3," Request for Comments RFC 1950, Internet Engineering Task Force, May 1996.
- [31] "LZMA SDK (Software Development Kit)," <https://7-zip.org/sdk.html>.
- [32] Y. Collet, "Finite State Entropy," 2013.
- [33] A. Robinson and C. Cherry, "Results of a prototype television bandwidth compression scheme," *Proceedings of the IEEE*, vol. 55, pp. 356–364, Mar. 1967.
- [34] M. Burrows, R. W. Taylor, and D. Wheeler, "A Block-sorting Lossless Data Compression Algorithm," 1994.
- [35] J. L. Bentley, D. D. Sleator, R. E. Tarjan, and V. K. Wei, "A locally adaptive data compression scheme," *Commun. ACM*, vol. 29, pp. 320–330, Apr. 1986.
- [36] F. Alted, "Blosc: A blocking, shuffling and loss-less compression library that can be faster than 'memcpy()'," 2018.
- [37] J. Teuhola, "A compression method for clustered bit-vectors," *Information Processing Letters*, vol. 7, pp. 308–311, Oct. 1978.
- [38] A. Trotman, M. Albert, and B. Burgess, "Optimal Packing in Simple-Family Codecs," in *Proceedings of the 2015 International Conference on The Theory of Information Retrieval*, (Northampton Massachusetts USA), pp. 337–340, ACM, Sept. 2015.
- [39] W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. Boston, Mass.: Kluwer, 8. print. 2004, digital duplication ed., 2004.
- [40] M. Loncaric, N. Jeppesen, and B. Zinberg, "Pcodec: Better Compression for Numerical Sequences," Feb. 2025.
- [41] D. Lemire and L. Boytsov, "Decoding billions of integers per second through vectorization," *Softw. Pract. Exper.*, vol. 45, pp. 1–29, Jan. 2015.
- [42] T. Pelkonen, S. Franklin, J. Teller, P. Cavallaro, Q. Huang, J. Meza, and K. Veeraraghavan, "Gorilla: A fast, scalable, in-memory time series database," *Proc. VLDB Endow.*, vol. 8, pp. 1816–1827, Aug. 2015.
- [43] R. Rice and J. Plaunt, "Adaptive Variable-Length Coding for Efficient Compression of Spacecraft Television Data," *IEEE Transactions on Communication Technology*, vol. 19, pp. 889–897, Dec. 1971.
- [44] A. Apostolico and A. Fraenkel, "Robust transmission of unbounded strings using Fibonacci representations," *IEEE Transactions on Information Theory*, vol. 33, pp. 238–245, Mar. 1987.
- [45] Welch, "A Technique for High-Performance Data Compression," *Computer*, vol. 17, pp. 8–19, June 1984.
- [46] J. G. Matt, "QuaRs: A Transform for Better Lossless Compression of Integers," Jan. 2025.
- [47] P. Elias, "Universal codeword sets and representations of the integers," *IEEE Trans. Inform. Theory*, vol. 21, pp. 194–203, Mar. 1975.
- [48] H. A. Dau, E. Keogh, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, Yanping, B. Hu, N. Begum, A. Bagnall, A. Mueen, G. Batista, and Hexagon-ML, "The UCR time series classification archive," Oct. 2018.
- [49] A. Reiss, "PAMAP2 Physical Activity Monitoring," 2012.
- [50] S. Makonin, "AMPds2: The Almanac of Minutely Power dataset (Version 2)," 2016.
- [51] S. Fothergill, H. Mentis, P. Kohli, and S. Nowozin, "Instructing people for training gestural interactive systems," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, (Austin Texas USA), pp. 1737–1746, ACM, May 2012.